ขนาดของเครือข่าย

ระยะ	ชนิด	ตัวอย่าง
~1 ม.	PAN	Bluetooth piconet
10 ม.–1 กม.	LAN	Ethernet, Wi-Fi
~10 กม.	MAN	เคเบิล, 802.16
100 กม.+	WAN	ใยแก้วนำแสง, 5G
~10,000 กม.	Internet	TCP/IP เครือข่ายของเครือข่าย

อินเทอร์เน็ตไม่ใช่เครือข่ายเดียว แต่คือ**เครือข่ายที่เชื่อมกันด้วยโพรโทคอล** เครือข่ายภูมิภาค องค์กร และมือถือ มาบรรจบที่โครงข่ายหลัก (Global ISP)

แพ็กเก็ตเดินผ่านที่ไหน

- **PAN:** master + slave (หูฟัง นาฬิกา รถ) Bluetooth แย่งคลื่นระยะสั้น
- **บ้าน:** เกตเวย์ออก ISP, โมเด็ม DSL/เคเบิล, ทั้งทราฟฟิกยืดหยุ่น (เว็บ) และสตรีมที่อ่อนไหวต่อเวลา, เซ็นเซอร์ IoT
- **NAT:** โฮสต์ภายในหลายเครื่อง (10.0.0.1–3) ใช้ IP สาธารณะตัวเดียว (เช่น 138.76.29.7) IP ภายในมีความหมายแค่ในบ้าน
- **LAN องค์กร:** เราเตอร์สถาบัน, สวิตช์ core/edge, 1–10 Gbps, แชร์เซิร์ฟเวอร์ สวิตช์แยก collision domain
- **VLAN (เกริ่น):** สายเดิม แต่จัดกลุ่มตามแผนกหรือความปลอดภัย ไม่ตามห้อง

รูปของเครือข่ายไร้สาย

- **Wi-Fi infrastructure:** สถานีผ่าน AP เข้าสาย **Ad hoc:** สถานีคุยกันตรง ไม่มี AP แชร์ช่องสัญญาณเดียวกัน
- **เซลลูลาร์:** UE ↔ สถานีฐาน (eNodeB) ↔ core การโอนย้าย: (1) เกาะเสาปัจจุบัน (2) วัดเสาข้างเคียงและเตรียม (3) ส่งมอบเซสชันโดยสตรีมไม่ขาด
- **ศูนย์ข้อมูล:** core router, edge switch, แร็ค Load balancing กระจายงาน ใช้กับคลาวด์ (Netflix, Google)

มีสาย vs ไร้สาย

	มีสาย	ไร้สาย
ตัวกลาง	ทองแดง, ไฟเบอร์	คลื่นวิทยุ
การชน	สวิตช์แยกช่องได้	ใช้สื่อร่วม ต้องมีกฎเข้าใช้
การเคลื่อนที่	อยู่กับที่	มีการโอนย้าย
เทคโนโลยี	Ethernet, ไฟเบอร์	802.11, 4G/5G, Bluetooth

หนึ่งวันของแพ็กเก็ต: โทรศัพท์ (5G) → ขอบเซลล์ → WAN ใยแก้ว → load balancer ในศูนย์ข้อมูล → เซิร์ฟเวอร์ที่เล่นวิดีโอ

เทคโนโลยีเข้าถึงเครือข่าย

เทค	ที่	การใช้	อัตรา
FTTH	บ้าน, ไฟเบอร์	เฉพาะ	100 Mbps–1 Gbps+
HFC	บ้าน, โคแอกซ์	ใช้ร่วม	หลัก 10–100 Mbps
DSL	บ้าน, ทองแดง	เฉพาะ	<10–หลัก 10 Mbps
Ethernet	องค์กร	เฉพาะ	10 Mbps–100 Gbps
Wi-Fi	บ้าน/ที่ทำงาน	ร่วม, ไม่ต้องใบอนุญาต	สูง, ระยะสั้น
4G/5G	พื้นที่กว้าง	ร่วม, มีใบอนุญาต	ปานกลาง–สูง, เคลื่อนที่

Circuit vs Packet switching

	Circuit	Packet
เส้นทาง	จองตลอดสาย (สล็อต TDM)	ใช้ร่วม แบบ store-and-forward
เหมาะกับ	อัตราคงที่ที่การันตี	ข้อมูลเป็นช่วง ๆ (bursty)
ราคาที่จ่าย	ช่องว่างถูกทิ้ง	คิวหน่วง และอาจสูญแพ็กเก็ต
ตัวอย่าง	ลิงก์ 2 Mbps ผู้ใช้คนละ 1 Mbps ได้ไม่เกิน 2 สาย	ลิงก์เดียวกันรับ 3 คนได้ ถ้าไม่ส่งพร้อมกัน

ใครถือโครงข่ายหลัก

- **Access ISP → regional ISP → tier-1** ระดับ tier-1 เชื่อมกันแบบ peering
- **Peering** (มักผ่าน **IXP**) เพื่อไม่ต้องจ่ายค่า transit ให้ ISP ระดับล่าง
- **เครือข่ายผู้ให้เนื้อหา** (เช่น Google) สร้างทางของตัวเองและแคชใกล้ผู้ใช้ เพื่อเลี่ยงค่า transit

โฮสต์และมาตรฐาน

โฮสต์ / end system คืออุปกรณ์ที่รันแอปบนเครือข่าย: PC, โทรศัพท์, แร็คเซิร์ฟเวอร์, เซ็นเซอร์ IoT **โพรโทคอล** คือข้อตกลงเรื่องรูปแบบ ลำดับ และการกระทำของข้อความ มาตรฐาน (HTTP, TCP, …) ทำให้ผู้ผลิตต่างค่ายคุยกันได้ แพ็กเก็ตมีเพย์โหลด เส้นทางเพิ่มความหน่วง และภัยคุกคามอยู่บนเส้นทางเดียวกัน

เพิ่มเติม ลิงก์เฉพาะ (FTTH, DSL, Ethernet แบบสวิตช์) ไม่แชร์ฮอปล่าสุด HFC กับ Wi-Fi แชร์ เพื่อนบ้านจึงทำให้ช้าได้ แม้ตัวเลขบนโมเด็มจะสูง

ความหน่วง 4 ชนิด

d_end-end = d_proc + d_queue + d_trans + d_prop
d_proc: ตรวจเฮดเดอร์ แล้วตัดสินใจส่งต่อ
d_queue: รอในบัฟเฟอร์
d_trans = **L / R** (เวลาผลักบิตลงสาย)
d_prop = **d / s** (สัญญาณเดินทาง; ไฟเบอร์ s ≈ 2×10^8 m/s)

Transmission คือด่านเก็บเงิน (นานแค่ไหนกว่าจะปล่อยรถลงถนน) Propagation คือการขับ (ระยะ/ความเร็ว) คนละตัวแปรกัน

โจทย์จากสไลด์

- L = 4,000 บิต, R = 2 Mbps → d_trans = 4000/2×10^6 = **2 ms**
- รูป 5 MB, อัปสตรีม HFC 10 Mbps ใช้ 1 MB = 10^6 ไบต์ → L = 4×10^7 บิต d_trans = 4×10^7/10^7 = **4 วินาที**
- 1,000 ไบต์, 2,000 กม., 10 Mbps, s = 2×10^8 m/s
  d_trans = 8,000/10^7 = **0.8 ms**
  d_prop = 2×10^6 / 2×10^8 = **10 ms** การเดินทางนานกว่าการผลักบิต

ทรูพุตและการสูญหาย

Throughput = **min(R1, R2, R3, …)**
ตัวอย่าง: 2 Mbps | 100 kbps | 1 Mbps → **100 kbps**

ความเข้มทราฟฟิก **I = L a / R**
a = อัตราแพ็กเก็ตเข้าเฉลี่ย ถ้า **I > 1** คิวโตไม่หยุด บัฟเฟอร์เต็ม แพ็กเก็ตใหม่ถูกทิ้ง ความหน่วงพุ่งเมื่อ I เข้าใกล้ 1

เพิ่มเติม กับดักหน่วย: **Mb ≠ MB** (คูณ 8) สไลด์ใช้ 10^6 ไม่ใช่ 2^20 เว้นแต่โจทย์บอกไว้ d_trans ไม่ขึ้นกับระยะ d_prop ไม่ขึ้นกับ L และ R ความหน่วงต่อโหนดคือสี่พจน์นี้ ปลายทางถึงปลายทางคือผลรวมตลอดเส้นทาง และคิวคือพจน์ที่เปลี่ยนตามโหลด

สแตก 5 ชั้น

ชั้น	หน้าที่	ชื่อข้อมูล
5 Application	HTTP, SMTP, DNS	Message
4 Transport	โปรเซสถึงโปรเซส	Segment
3 Network	โฮสต์ถึงโฮสต์, หาเส้นทาง	Datagram
2 Link	หนึ่งฮอป โหนดที่ติดกัน	Frame
1 Physical	สัญญาณบนตัวกลาง	Bits

การห่อหุ้ม (encapsulation)

แต่ละชั้นเติมเฮดเดอร์ สวิตช์ดู**เฮดเดอร์ลิงก์อย่างเดียว** แล้วส่งเฟรมต่อ เราเตอร์ถอดเฟรม อ่าน**เฮดเดอร์ IP** แล้วสร้าง**เฟรมใหม่**สำหรับลิงก์ถัดไป IP ต้นทางและปลายทางไม่เปลี่ยนข้ามฮอป MAC กับ FCS เปลี่ยนทุกฮอป เปรียบเทียบ: ตั๋ว (L3) บอกเมืองปลายทาง พาหนะ (L2) เป็นของท้องถิ่น

ทำไมต้องหั่นข้อความ

ส่ง 10^6 บิต ผ่าน 3 ลิงก์ ที่ 5 Mbps ทั้งก้อนใช้ 0.2 วินาทีต่อลิงก์ Store-and-forward ทั้งก้อน: **3 × 0.2 = 0.6 วินาที**
หั่นเป็น 100 แพ็กเก็ต ใบละ 10^4 บิต (ใบละ 0.002 วินาที) เวลาแบบไปป์ไลน์ = (3 + 100 − 1) × 0.002 = **0.204 วินาที**

ภัยบนเส้นทาง

- **มัลแวร์ที่แพร่เอง** (เวิร์ม): กระจายโฮสต์สู่โฮสต์ โดยผู้ใช้ไม่ต้องเปิดไฟล์
- **บอตเน็ต / DDoS:** โฮสต์ที่ถูกยึดจำนวนมากยิงขยะใส่เป้าเดียวกันพร้อมกัน
- **MITM:** Trudy นั่งระหว่าง Alice กับ Bob อ่านหรือแก้แพ็กเก็ตได้

เพิ่มเติม Packet switching มีไว้เพื่อไม่ให้การส่งก้อนใหญ่แช่แข็งเราเตอร์ทุกตัวจนกว่าวัตถุทั้งก้อนจะข้าม ราคาที่จ่ายคือเฮดเดอร์ การประกอบกลับ และการที่แพ็กเก็ตแต่ละใบอาจหาย Wireshark บนพีซีปกติไม่แสดง preamble, SFD และ FCS แม้บนสายจะมี

แอปถูกสร้างอย่างไร

- **Client–server:** ไคลเอนต์ขอ เซิร์ฟเวอร์ที่เปิดตลอดตอบ (เว็บ เมล ส่วนควบคุมของสตรีม)
- **P2P:** ไม่มีศูนย์ที่เปิดตลอด แต่ละเพียร์เป็นทั้งไคลเอนต์และเซิร์ฟเวอร์ (BitTorrent) เพียร์ยิ่งมาก ความจุยิ่งเพิ่มได้
- แอปที่ควรเรียกชื่อได้: เว็บ เมล วิดีโอสตรีม VoIP เกม ค้นหา ล็อกอินระยะไกล แชต

โปรเซสและซ็อกเก็ต

โปรเซส คือโปรแกรมที่กำลังรัน โฮสต์เดียวกันคุยผ่าน OS คนละโฮสต์ส่งข้อความผ่าน**ซ็อกเก็ต** (ประตู) จะรับข้อความได้ โปรเซสต้องมีตัวระบุ: **ที่อยู่ IP + หมายเลขพอร์ต** IP หาเครื่อง พอร์ตหาโปรเซส

แอปขออะไรจากทรานสปอร์ต

	TCP	UDP
การส่ง	สตรีมไบต์ที่เชื่อถือได้	ดาตาแกรม best-effort
การตั้งต้น	แฮนด์เชค	ไม่มี
อื่น ๆ	ควบคุมการไหลและความแออัด	ไม่การันตีลำดับ การถึง หรือจังหวะเวลา
ใช้กับ	HTTP, SMTP, IMAP	คิวรี DNS, สตรีมบางแบบ / ฐานของ QUIC

แอปกับทรานสปอร์ตโดยทั่วไป: ถ่ายไฟล์และเว็บ → TCP, SMTP → TCP, DNS → UDP (TCP เมื่อข้อมูลใหญ่หรือถ่ายโอนโซน), DASH → HTTP/TCP หรือ HTTP/3/QUIC

พอร์ตที่ควรจำ

พอร์ต	บริการ
20 / 21	FTP data / control
22	SSH / SFTP
23	Telnet (ข้อความเปล่า)
25	SMTP
53	DNS (UDP และ TCP)
80 / 443	HTTP / HTTPS, QUIC
110 / 143	POP3 / IMAP
465, 587	SMTPS / submission
993 / 995	IMAPS / POP3S

TLS: ซ็อกเก็ต TCP/UDP เปล่าส่งข้อความชัด TLS (บน TCP: HTTPS, IMAPS, POP3S) เข้ารหัส QUIC ใส่การเข้ารหัสไว้ในทรานสปอร์ตที่วิ่งบน UDP

เว็บและ HTTP

หน้าเว็บคือ**วัตถุ** (HTML, JPEG, …) แต่ละชิ้นมี URL อาจอยู่คนละเซิร์ฟเวอร์ HTTP คือโพรโทคอลชั้นแอป: ไคลเอนต์/เซิร์ฟเวอร์ พอร์ต 80 ปกติ ใช้ TCP ตัว HTTP **ไร้สถานะ** (เซิร์ฟเวอร์ไม่จำคำขอเก่า เว้นแต่คุกกี้มาช่วย)

การเชื่อมต่อ

- **Non-persistent:** หนึ่งการเชื่อมต่อ TCP ส่งได้อย่างมากหนึ่งวัตถุ เปิด ส่ง ปิด ราคา ≈ **2 RTT + เวลาส่งไฟล์** ต่อวัตถุ (แฮนด์เชค + ขอ/ตอบ) และมีโอเวอร์เฮดของ OS ทุกครั้ง
- **Persistent (HTTP/1.1):** ตั้ง TCP ครั้งเดียว (1 RTT) แล้วส่งหลายวัตถุบนสายนั้น วัตถุละประมาณ **1 RTT** บวกเวลาส่ง เซิร์ฟเวอร์อาจเปิดสายค้างไว้

> RTT = แพ็กเก็ตเล็กไปเซิร์ฟเวอร์แล้วกลับ หน้าแบบ non-persistent ที่มี HTML 1 ไฟล์ + รูป 10 รูป ≈ 11 × (2 RTT + เวลาส่ง) ถ้าเปิดทีละสาย

ข้อความ

คำขอ: เมธอด, URL, เวอร์ชัน แล้วตามด้วยเฮดเดอร์ บรรทัดว่าง บอดี้ถ้ามี
คำตอบ: เวอร์ชัน, รหัสสถานะ, วลี, เฮดเดอร์, บรรทัดว่าง, บอดี้

เมธอด	ความหมาย
GET	ขอวัตถุ พารามิเตอร์อยู่ใน URL
POST	ส่งฟอร์ม / บอดี้ไปเซิร์ฟเวอร์
HEAD	เอาเฉพาะเฮดเดอร์ ไม่มีบอดี้
PUT / DELETE	อัปโหลด / ลบวัตถุ

สถานะ: **200 OK**, **301** Moved, **304** Not Modified, **400** Bad Request, **404** Not Found, **505** เวอร์ชัน HTTP ไม่รองรับ กลุ่ม: 2xx สำเร็จ, 3xx เปลี่ยนทาง, 4xx ฝั่งไคลเอนต์, 5xx ฝั่งเซิร์ฟเวอร์

คุกกี้

HTTP ไร้สถานะ คุกกี้เติมความจำสี่ส่วน: (1) `Set-Cookie` ในคำตอบ (2) เบราว์เซอร์เก็บรหัส (3) คำขอถัดไปส่ง `Cookie` (4) เซิร์ฟเวอร์ใช้รหัสไปค้นในฐานข้อมูล

- ใช้ทำ: เซสชันล็อกอิน ตะกร้า ค่าที่ชอบ การติดตาม / โฆษณา
- **ฝ่ายแรก:** เว็บที่กำลังเปิด **ฝ่ายที่สาม:** โดเมนอื่น (เครือข่ายโฆษณา) ติดตามข้ามเว็บ ตัวอย่าง: หน้า NYTimes โหลด AdX คุกกี้ 1634 จำว่า “sports” วันถัดไปจำ “arts”
- Firefox บล็อกคุกกี้ฝ่ายที่สามเป็นค่าเริ่มต้น (ETP) **GDPR:** รหัสคุกกี้และ IP ระบุตัวบุคคลได้

แคช

- **แคชเบราว์เซอร์:** บนดิสก์หรือแรมของผู้ใช้
- **พร็อกซี / Web cache:** เซิร์ฟเวอร์ที่ใช้ร่วม เป็นเซิร์ฟเวอร์ของไคลเอนต์ และเป็นไคลเอนต์ของต้นทาง ลดโหลดลิงก์เข้าถึงและความหน่วง

ภาพในสไลด์: ลิงก์เข้าถึงเกือบเต็ม (การใช้ ≈ 0.97) ทำให้ความหน่วงปลายทางถึงปลายทางสูง (ความหน่วงใน LAN น้อยมาก) แคชที่ตอบได้บางส่วนจะลดการใช้ลิงก์เข้าถึงและความหน่วงเฉลี่ย

Conditional GET

ไคลเอนต์ส่ง `If-Modified-Since: <วันที่>` ไม่เปลี่ยน → **304 Not Modified** ไม่มีบอดี้ เปลี่ยนแล้ว → **200 OK** พร้อมวัตถุใหม่

HTTP/1.1 → HTTP/2 → HTTP/3

- **ไปป์ไลน์ HTTP/1.1:** ส่ง GET หลายอันบน TCP เดียว แต่เซิร์ฟเวอร์ตอบตามลำดับ วัตถุใหญ่ขวางวัตถุเล็ก (**HOL blocking**) เซกเมนต์ TCP ที่หายจะหยุดทุกวัตถุบนสายนั้น
- **HTTP/2:** เมธอดและรหัสเดิม หั่นวัตถุเป็นเฟรมแล้วสลับส่ง (multiplexing) วัตถุเล็กไม่ต้องรอวัตถุใหญ่ แต่ยังเป็น TCP เดียว แพ็กเก็ตหายยังหยุดทั้งสาย
- **HTTP/3 = QUIC บน UDP:** การเชื่อมต่อ กู้แพ็กเก็ตหาย และควบคุมความแออัด อยู่ในยูเซอร์สเปซ สตรีมอิสระ แพ็กเก็ตหายสายหนึ่งไม่แช่แข็งสายอื่น แฮนด์เชครวมทรานสปอร์ตกับ TLS ได้ **0-RTT** ใช้ตั๋วเซสชัน TLS ส่งข้อมูลได้ตั้งแต่เที่ยวแรก

อีเมล

- สามชิ้น: **user agent**, **เมลเซิร์ฟเวอร์** (กล่องจดหมาย + คิวขาออก), **SMTP**
- SMTP (RFC 5321): TCP พอร์ต 25 เซิร์ฟเวอร์ผู้ส่งทำตัวเป็นไคลเอนต์ ส่งตรงถึงเซิร์ฟเวอร์ผู้รับ ขั้น: แฮนด์เชค (`220`, `HELO`, `250`), ส่งข้อความ (`354` เริ่มรับเมล), ปิด คำสั่งเป็น ASCII
- UA ของ Alice → เซิร์ฟเวอร์ของเธอ (SMTP หรือ HTTP) เซิร์ฟเวอร์เธอ → เซิร์ฟเวอร์ของ Bob (SMTP) Bob ดึงเมลด้วย IMAP, POP หรือเว็บ
- **RFC 5322 / 2822** คือตัวจดหมาย (เฮดเดอร์ `To:` `From:` `Subject:` บรรทัดว่าง บอดี้) คนละอย่างกับซอง SMTP (`MAIL FROM`, `RCPT TO`)
- **IMAP:** เมลอยู่บนเซิร์ฟเวอร์ มีโฟลเดอร์ ลบที่นั่น **POP3:** มักดาวน์โหลด (แล้วลบบนเซิร์ฟเวอร์) เว็บเมลคือ HTTP หน้า SMTP + IMAP/POP

DNS

คนใช้ชื่อ เราเตอร์ใช้ IP DNS คือฐานข้อมูลแบบกระจาย ไม่รวมศูนย์ เพราะจะเป็นจุดล้มจุดเดียว ทราฟฟิกแน่น ไกล และดูแลยาก

- ลำดับชั้น: **root** → **TLD** (.com, .org, .th, …) → **authoritative** (เซิร์ฟเวอร์ของโดเมนนั้น) → คำตอบ เครื่องถาม **local DNS** ก่อน
- **Iterative:** แต่ละเซิร์ฟเวอร์ชี้รีโซลเวอร์ไปตัวถัดไป (รีโซลเวอร์เดินเอง) **Recursive:** เซิร์ฟเวอร์ที่ถูกถามวิ่งหาชื่อให้ แล้วคืนคำตอบสุดท้าย
- แคชเก็บคู่ชื่อ–ที่อยู่พร้อม **TTL** คำตอบในแคชอาจเก่าจนกว่า TTL จะหมด

RR	ชื่อ → ค่า
A	ชื่อโฮสต์ → IP
NS	โดเมน → เซิร์ฟเวอร์ authoritative
CNAME	ชื่อเล่น → ชื่อจริง
MX	โดเมน → เมลเซิร์ฟเวอร์

ข้อความ DNS: เฮดเดอร์ (id, แฟล็ก), คำถาม, answer RR, authority RR, additional RR จดทะเบียนชื่อที่ registrar แล้วใส่เรคอร์ด NS และ A (กับ MX) การโจมตี: DDoS ที่ root/TLD, คำตอบปลอม, cache poisoning

วิดีโอและ CDN

- วิดีโอ = ภาพในอัตราคงที่ (เช่น 24 ภาพ/วินาที) **Spatial** บีบภายในเฟรม **Temporal** ส่งส่วนต่างระหว่างเฟรม มีทั้ง CBR และ VBR
- สตรีมที่บันทึกไว้: อัดก่อน แล้วส่ง ข้อจำกัดคือจังหวะเล่นต้องตรงต้นฉบับ ไคลเอนต์จึงบัฟเฟอร์เพื่อกลบ jitter
- **DASH:** หั่นไฟล์เป็นชิ้น แต่ละชิ้นเข้ารหัสหลายเรต ไฟล์ manifest บอก URL ไคลเอนต์เลือกเรตจากแบนด์วิดท์ที่วัดได้
- **CDN:** เซิร์ฟเวอร์ยักษ์เครื่องเดียวแพ้ทั้งระยะทางและโหลด **Enter deep** (เซิร์ฟเวอร์ในเครือข่ายเข้าถึงจำนวนมาก) หรือ **bring home** (ไซต์น้อยลง แต่ใหญ่ขึ้น) Netflix: manifest จากบริการวิดีโอจากโหนด OpenConnect

เพิ่มเติม ภาพซ็อกเก็ต ไม่ใช่โปรแกรมเต็ม **UDP:** ไม่แฮนด์เชค ไคลเอนต์ `sendto` / `recvfrom` ใส่ที่อยู่ทุกครั้ง **TCP:** เซิร์ฟเวอร์ `bind+listen+accept` ไคลเอนต์ `connect` แล้วทั้งคู่ อ่าน/เขียนสตรีมไบต์ แอปต้องระบุ IP และพอร์ตของอีกฝั่งเอง API ไม่เลือกให้

ชั้นลิงก์ทำอะไร

ชั้นกายภาพขยับบิต ชั้นลิงก์ส่งเฟรมข้าม**หนึ่งฮอป** แอปมองแบบปลายทางถึงปลายทาง ลิงก์ทำงานทีละฮอป ที่เราเตอร์ทุกตัว เฟรมเก่าถูกถอดแล้วสร้างเฟรมใหม่

- **LLC:** คุยกับชั้นเครือข่าย มัลติเพล็กซ์โพรโทคอล อาจมีการควบคุมการไหลและกู้ข้อผิดพลาด
- **MAC:** ตีกรอบ ที่อยู่ลิงก์ FCS และ (บนสื่อที่ใช้ร่วม) ใครส่งได้

ลิงก์	รูป	สิ่งที่เพิ่ม
Ethernet 802.3	LLC + MAC	เฟรม ที่อยู่ FCS
Wi-Fi 802.11	LLC + MAC	การเข้าใช้ ACK ส่งซ้ำ
Token Ring 802.5	LLC + MAC	โทเคน (ระบบเก่า)
802.15.4	Adapt + MAC	WPAN พลังต่ำ
PPP, HDLC	โพรโทคอลเดียว	ตีกรอบเอง ไม่แยก LLC/MAC แบบ 802
LAPB / X.25	แนว HDLC	ลิงก์ WAN ที่เชื่อถือได้
LTE / 5G	สแตกของตัวเอง	PDCP, RLC, MAC

ลิงก์เฉพาะ (พอร์ตสวิตช์ฟูลดูเพล็กซ์, จุดต่อจุด): ไม่แย่งสาย **ลิงก์ใช้ร่วม** (บัส ฮับ ไร้สาย): สองคนส่งพร้อมกันแล้วชน นี่คือโจทย์ MAC (บท 6.1 อยู่ชีทหน้า 4)

การตีกรอบ (framing)

เฟรมคือซอง: เฮดเดอร์ เพย์โหลด เทรลเลอร์ (มักเป็น FCS) และวิธีเห็นขอบ แบบทั่วไป: **Flag | Address | Control | Data | FCS | Flag**

	รายไบต์	รายบิต	ความยาว / PHY
ขอบ	ไบต์คั่น	บิตแฟล็ก	ฟิลด์ความยาว หรือซิงก์ของ PHY
โปร่งใส	byte stuffing	bit stuffing	ไม่ยัดตัวคั่น
ตัวอย่าง	BISYNC, SLIP, PPP แบบอะซิงก์	HDLC, PPP ซิงก์, Frame Relay, CAN	Ethernet, 802.11, ATM

- **Byte stuffing:** ถ้าไบต์แฟล็กโผล่ในข้อมูล ให้หลบด้วย ESC เพื่อไม่ถูกอ่านเป็นแฟล็ก แฟล็ก PPP คือ `0x7E` ยังเจอใน PPPoE (DSL/ไฟเบอร์) และลิงก์ WAN แบบอนุกรมบางระบบ
- **Bit stuffing (HDLC):** แฟล็ก = `01111110` ผู้ส่งแทรก 0 หลัง 1 ห้าตัวในข้อมูล ผู้รับดึง 0 นั้นออก ใช้บนอนุกรมซิงโครนัส (T1/E1, SONET) และ Cisco HDLC
- UART ใช้บิตเริ่ม/หยุด I²C ใช้เงื่อนไข START/STOP SPI ไม่มีเฟรมมาตรฐานเดียว **แพตติงของ Ethernet ไม่ใช่ stuffing** มันแค่เติมเฟรมสั้นให้ถึงความยาวขั้นต่ำ

ข้อผิดพลาด

บิตพลิกเพราะสัญญาณรบกวน สัญญาณรบกวนความร้อน สายหรือ NIC เสีย ฟ้าผ่า EMP **ผิดบิตเดียว** พลิกหนึ่งบิต **เบิร์สต์** พลิกเป็นช่วง (ภาพในสไลด์คือเบิร์สต์ 8 บิต)

คนทำได้สามทาง: เมิน ถามใหม่ หรือซ่อม เครือข่ายใช้สองทางหลังเป็น **ARQ** (ตรวจเจอ แล้วขอส่งใหม่) หรือ **FEC** (ใส่ส่วนเกินพอให้แก้ได้)

บล็อกโค้ด: ดาต้าเวิร์ด k บิต ส่วนเกิน r บิต โค้ดเวิร์ด n = k + r มีดาต้าเวิร์ดได้ 2^k แบบ และโค้ดเวิร์ดที่เป็นไปได้ถึง 2^n แบบ แต่ใช้จริงไม่ครบ ผู้รับคำนวณ**ซินโดรม** ซินโดรม 0 แปลว่าถือว่าถูก ของ Hamming ซินโดรมคือตำแหน่งบิตที่ผิด

พาริตี

พาริตีคู่: เติมบิตให้จำนวน 1 เป็นคู่ พาริตีคี่: ให้เป็นคี่ วงจรสร้างและตรวจคือ XOR ตัวเดียวกัน ตัวอย่างคำว่า world ใน ASCII 7 บิต บวกพาริตีคู่ส่งเป็นหน่วย 8 บิต บิตพลิกแล้วซินโดรมเป็น 1

พาริตีธรรมดาจับผิดบิตเดียวได้ทุกกรณี และจับเบิร์สต์ได้ก็ต่อเมื่อจำนวนบิตเสียในหน่วยนั้นเป็น**เลขคี่** พลิกสองบิตในหน่วยเดียวกันจะมองไม่เห็น

พาริตีสองมิติ: พาริตีทุกแถวและทุกคอลัมน์ บิตเสียหนึ่งตำแหน่งจะทำให้แถวหนึ่งและคอลัมน์หนึ่งไม่ผ่าน จึง**แก้**ตำแหน่งนั้นได้ ส่งทีละแถว พร้อมบิตพาริตี

เช็กซัม (one's complement)

หั่นข้อความเป็นหน่วย m บิต บวกแบบมีตัวทดวนปลาย แล้วส่งส่วนเติมเต็ม ผู้รับบวกข้อความ + เช็กซัม ถ้าส่วนเติมเต็มของผลบวกเป็น 0 แสดงว่าสะอาด เช็กซัมเฮดเดอร์ IPv4 คลุมเฉพาะเฮดเดอร์ เช็กซัม TCP/UDP คลุมเฮดเดอร์ + ข้อมูล ลิงก์ชอบ CRC มากกว่าเพราะจับเบิร์สต์ได้ดี

> 10101001 + 00111001 = 11100010 เช็กซัม = **00011101**
> ส่ง: 10101001 00111001 00011101 ผลบวกสามส่วน = 11111111 → ส่วนเติมเต็ม **00000000** (ผ่าน)
> เบิร์สต์ยาว 5 ที่พลิก 4 บิต ทำให้ซินโดรมไม่เป็นศูนย์ → ทิ้ง

CRC

มองข้อมูลเป็นพหุนาม ตกลงตัวหารกัน เติมศูนย์ท้าย r ตัว (เท่าดีกรี) แล้วหารแบบโมดูโล-2 (XOR ไม่ยืม) เศษคือ CRC สั้นกว่าตัวหารหนึ่งบิต โค้ดเวิร์ด = ข้อมูล + เศษ หารตัวหารลงตัว ผู้ตรวจหารอีกครั้ง เศษไม่เป็นศูนย์แปลว่าผิด

> CRC(7,4) ในสไลด์ ตัวหาร **1011** เศษ **110** โค้ดเวิร์ดลงท้ายด้วย 3 บิตนี้ การลบแบบโมดูโล-2 คือ XOR

Hamming (พาริตีคู่)

บิตพาริตีอยู่ที่กำลังของ 2: r1, r2, r4, r8, … แต่ละตัวตรวจตำแหน่งที่ดัชนีมีบิตนั้นติด ข้อมูล **1001101** วางเป็น d7…d1 คั่นบิตพาริตี ได้โค้ดเวิร์ด **10011100101**

> ตำแหน่ง 11→1: d7 d6 d5 r8 d4 d3 d2 r4 d1 r2 r1
> = 1 0 0 1 1 1 0 0 1 0 1
> รับมา 10010100101 ตรวจ: r1=1, r2=1, r4=1, r8=0 ซินโดรม r8 r4 r2 r1 = 0111 = **7** พลิกบิตที่ 7

Interleaving: สลับบิตของหลายโค้ดเวิร์ด เบิร์สต์จะโดนโค้ดเวิร์ดละหนึ่งบิต แล้ว Hamming แก้ทีละคำได้

> **เพิ่มเติม** พาริตีพลาดเมื่อจำนวนบิตพลิกเป็นเลขคู่ เช็กซัมพลาดเบิร์สต์ ได้หลายแบบ CRC (FCS) คือสิ่งที่ Ethernet, Wi-Fi, HDLC และ PPP ใช้จริงบนลิงก์ การแก้ผิด (Hamming, interleaving) ใช้เมื่อส่งใหม่แพงหรือทำไม่ได้

การควบคุมการไหล

ผู้ส่งต้องไม่เร็วกว่าผู้รับ ทั้งสองฝั่งมีบัฟเฟอร์ ถ้าบัฟเฟอร์ผู้รับเต็ม เฟรมหายได้แม้สายจะดี

- **โพรโทคอลธรรมดา:** ไม่ควบคุมการไหล ไม่ควบคุมข้อผิดพลาด ไม่มีเลขลำดับ ส่งเมื่อมีข้อมูล
- **Stop-and-wait ARQ:** ส่งหนึ่งเฟรม รอ ACK (ผู้รับตรวจ CRC) เลขลำดับมีแค่ **0 กับ 1** เพื่อไม่ให้เฟรมที่ส่งซ้ำถูกนับเป็นเฟรมใหม่

กรณีของ stop-and-wait: ACK ปกติ, **เฟรมหาย** → ไทม์เมอร์ ส่งเลขเดิมซ้ำ, **ACK หาย** → ผู้ส่งส่งซ้ำ ผู้รับเห็นเลขซ้ำจึงทิ้งแต่ ACK ใหม่, **ACK มาช้า** → เลข 0/1 แยก ACK เก่าออกจาก ACK ของเฟรมถัดไป

ไปป์ไลน์

มีหลายเฟรมอยู่ระหว่างทางพร้อมกัน การใช้ลิงก์ดีขึ้นบนลิงก์ยาวหรือเร็ว มีสองแบบ: Go-Back-N และ Selective Repeat

	Go-Back-N	Selective Repeat
เลข m บิต	มี 2^m ค่า	มี 2^m ค่า
หน้าต่าง	Ws ≤ 2^m−1, Wr = 1	Ws = Wr ≤ 2^(m−1)
m = 4	Ws = 15 (ค่าสูงสุดเป็นคี่), Wr = 1	W = 8 ทั้งสองฝั่ง (ครึ่งหนึ่ง)
ผู้รับ	รับเฉพาะเฟรมถัดไป (Rn) ที่เหลือทิ้ง	บัฟเฟอร์เฟรมที่มาก่อนลำดับ
ACK	สะสม (cumulative)	รายเฟรม
ไทม์เมอร์	ตัวเดียว ที่เฟรมค้างเก่าสุด	หนึ่งตัวต่อเฟรมที่ค้าง
ส่งซ้ำ	เฟรมที่หายและทุกเฟรมหลังมัน	เฉพาะเฟรมที่หายหรือเสีย

Stop-and-wait คือ GBN ที่ Ws = 1 และเลขเป็น {0,1} ACK ของ GBN ที่หาย อาจถูกคลุมด้วย ACK สะสมตัวถัดไป ส่งสำเร็จ 20 แพ็กเก็ต เลขจะวน: 0…15, 0, 1, 2, 3

> **เพิ่มเติม** ถ้า GBN ใช้ Ws = 2^m หรือ SR ใช้หน้าต่างใหญ่กว่าครึ่งของเซตเลขลำดับ ผู้รับจะแยกเฟรมใหม่จากเฟรมส่งซ้ำไม่ได้ตอนเลขวน นั่นคือเหตุผลที่ขีดจำกัดเข้ม

สมรรถนะ (จากสไลด์)

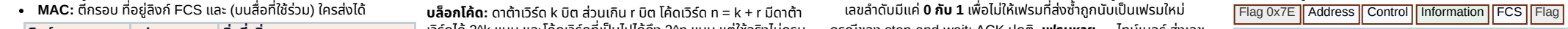
- ระยะ 1000 ม. ความเร็ว 500 ม./วินาที ไม่นับเวลาส่ง RTT = 2×1000/500 = **4 วินาที**
- เฟรม 100 บิต รวม 1000 บิต → 10 เฟรม Stop-and-wait: 10 × 4 = **40 วินาที**
- ลิงก์เดียวกัน ส่งพร้อมกันได้ 2 เฟรม: 5 รอบ × 4 = **20 วินาที**

> **Bandwidth–delay product** = R × RTT = จำนวนบิตที่ส่งออกไปได้ระหว่างรอ ACK
> 1 Mbps × 20 ms = **20,000 บิต** เฟรม 1000 บิต → การใช้ลิงก์แบบ stop-and-wait = 1000/20000 = **5%**
> GBN ที่หน้าต่าง 15 เฟรม: 15,000/20,000 = **75%** (น้อยลงถ้าต้องส่งซ้ำ)

HDLC

มาตรฐานแบบรายบิต สำหรับลิงก์อนุกรมจุดต่อจุดและหลายจุด มีแฟล็ก bit stuffing, FCS (CRC 2 หรือ 4 ไบต์) และฟิลด์ควบคุมสำหรับโฟลว์กับข้อผิดพลาด

บริการลิงก์**ไร้การเชื่อมต่อ:** แต่ละเฟรมอยู่ได้เอง **มีการเชื่อมต่อ:** ตั้งโหมด ส่งข้อมูล แล้วปล่อย



เฟรม	ควบคุมขึ้นต้น	หน้าที่
I	0	ข้อมูลผู้ใช้ + ACK แบบพ่วง N(S) = เลขเฟรมนี้ N(R) = เลขที่อยากได้ถัดไป (ACK)
S	10	ควบคุมโฟลว์/ข้อผิดพลาดอย่างเดียว เมื่อพ่วง ACK ไม่ได้ N(R) เป็น ACK หรือ NAK
U	11	จัดการเซสชัน รหัสแยกคนละฝั่งของบิต P/F

บิต P/F: Poll เมื่อฝ่ายหลักส่ง (สั่งฝ่ายรองให้ตอบ) Final เมื่อฝ่ายรองส่ง (จบคำตอบของตน)

รหัสเฟรม S (2 บิต): **00 RR** ผู้รับพร้อม (ACK ส่งต่อได้) **01 RNR** ACK แต่ไม่ว่าง ชะลอ **10 REJ** NAK ของ Go-Back-N (ส่งใหม่ตั้งแต่ N(R)) **11 SREJ** NAK ของ Selective Repeat (ส่งใหม่แค่เฟรมนั้น)

คำสั่ง / คำตอบเฟรม U: SNRM, SNRME (normal response, แบบขยาย), SABM, SABME (asynchronous balanced), UP, UI, UA (ตอบรับ), RD, DISC, DM, RIM, SIM, RSET, XID, FRMR (ปฏิเสธเฟรม)

ตัวอย่างตั้งการเชื่อมต่อ: ฝ่ายหลัก SNRM หรือ SABM → ฝ่ายรอง UA → เฟรม I ปล่อยสาย: DISC → UA ภาพการพ่วง ACK: "ส่งของฉัน 0 อยากได้ของคุณ 2" แล้ว "ส่งของฉัน 1 อยากได้ของคุณ 2" แล้ว RR "อยากได้ของคุณ 3"

เฟรมญาติ

- **PPP:** แฟล็ก 0x7E, address 0xFF, control 0x03, ฟิลด์โพรโทคอล, ข้อมูล, FCS ใช้ byte stuffing LCP ตั้งค่าลิงก์ NCP ตั้งค่าโพรโทคอลเครือข่าย (มักเป็น IP)
- **Cisco HDLC:** เหมือน HDLC แต่เพิ่มฟิลด์ชนิดที่เป็นกรรมสิทธิ์ เพื่อให้หลายโพรโทคอลเครือข่ายใช้ลิงก์อนุกรมเดียวกัน
- **LAPB** คือลิงก์ที่เชื่อถือได้ซึ่งต่อจาก HDLC ใต้ X.25 Frame Relay กับ ATM เป็นการตีกรอบ WAN ที่เกี่ยวข้อง ATM ใช้เซลล์ขนาดคงที่แทนแฟล็ก

> **เพิ่มเติม** Normal response: ฝ่ายหลักเท่านั้นที่เริ่มส่ง ฝ่ายรองพูดเมื่อถูกโพลล์ Asynchronous balanced: ทั้งสองฝ่ายส่งเองได้ (แบบเราเตอร์สองตัว) คำว่า extended (SNRME, SABME) หมายถึงฟิลด์เลขลำดับยาวขึ้น (7 บิต → หน้าต่างใหญ่ขึ้น) Ethernet **ไม่**ใช้หน้าต่างแบบ HDLC มันไร้การเชื่อมต่อ และฝากความเชื่อถือไว้ที่ TCP

ใครส่งได้

ตระกูล	กฎ	ตัวอย่าง
Contention	แย่งกัน ง่าย แต่พังเมื่อแน่น	ALOHA, CSMA/CD, CSMA/CA
Controlled	ผลัดกัน	จองคิว, โพลล์, โทเคน
Channelization	แบ่งตัวกลาง	FDMA, TDMA, CDMA

การชนคือมีสัญญาณสองทางขึ้นไปบนสื่อร่วมพร้อมกัน (บัสหรือฮับ) ระดับสัญญาณเพี้ยน สวิตช์ฟูลดูเพล็กซ์ไม่มีการชน

ALOHA

ส่งทันทีที่มีเฟรม (ไร้สายยุคแรก 9600 bps) ไม่ฟังสื่อก่อน ชนแล้วสุ่ม backoff แล้วลองใหม่ **Pure:** ส่งเมื่อไรก็ได้ ช่วงเสี่ยง **2T** ทรูพุตสูงสุด ~18% **Slotted:** ส่งได้แค่ต้นสล็อต ต้องซิงก์ ช่วงเสี่ยง **T** สูงสุด ~37%

CSMA

ฟังก่อน ถ้าว่างจึงส่งได้ ยังชนได้: สองสถานีได้ยินว่าว่าง แล้วสัญญาณมาชนกันหลังความหน่วงการเดินทาง

- **Non-persistent:** ถ้าไม่ว่าง รอสุ่ม แล้วฟังใหม่ ชนน้อยลง แต่มีช่องว่างมากขึ้น
- **1-persistent:** ถ้าไม่ว่าง ให้ฟังต่อ ส่งทันทีที่ว่าง โลภ สถานีชนกันตอนเฟรมยาวจบ Ethernet รุ่นคลาสสิก
- **p-persistent:** ถ้าว่าง ส่งด้วยความน่าจะเป็น p ไม่งั้นรอหนึ่งสล็อต p = 0.2 คือโอกาสส่งในสล็อตนี้ 20%

CSMA/CD (Ethernet สายที่ใช้ร่วม)

ฟัง ส่ง และ**ฟังต่อระหว่างส่ง** ชนแล้วหยุด ส่ง jam แล้วลองใหม่ หลังชนครั้งที่ n ให้ k = min(n, 10) สุ่ม r จาก 0 … 2^k−1 รอ r × สล็อต สล็อต ≥ 2 T_prop,max สล็อต Ethernet 10 Mbps = 512 บิต-ไทม์ = **51.2 µs** ยอมแพ้หลังชนมากเกินไป (16 ครั้ง)

CSMA/CA (802.11)

ผู้ส่งไร้สายไม่ได้ยินการชนขณะตัวเองส่ง และสถานีที่ซ่อนอยู่อาจไม่ได้ยินอีกฝ่าย จึงเลี่ยงการชน ไม่ใช่ตรวจการชน

- รอ **DIFS** (หรือ **SIFS** ที่สั้นกว่า สำหรับ ACK, CTS และเฟรมสำคัญ) แล้ว backoff สุ่มที่นับเฉพาะตอนช่องว่าง
- **CW backoff** สุ่มว่าจะรอกี่สล็อต (CW = 15 จับได้ 7 → รอ 7) คนละแบบกับ p-persistent ที่ทอยเหรียญทุกสล็อต
- ผู้รับ ACK หลัง SIFS ไม่มี ACK → ถือว่าหาย ขยาย CW แล้วลองใหม่
- **NAV:** เฟรมใส่ Duration สถานีอื่นชะลอแม้จะไม่ได้ยินข้อมูล (virtual carrier sense)
- **RTS/CTS** (ทางเลือก, DCF): จองสื่อ เพื่อให้สถานีที่ซ่อนอยู่ได้ยิน CTS แล้วตั้ง NAV
- **EDCA:** เสียง/วิดีโอรอสั้นกว่าทราฟฟิกทั่วไป

Wi-Fi 4 (802.11n): MIMO, รวมช่อง Wi-Fi 5 (ac): DL MU-MIMO, ช่องกว้างขึ้น Wi-Fi 6 (ax): OFDMA + UL/DL MU-MIMO Wi-Fi เป็นเครื่องหมายการค้า IEEE 802.11 คือมาตรฐาน

ผลัดกัน และแบ่งช่อง

- **จองคิว:** มินิสล็อตจองหนึ่งช่องต่อสถานี แล้วตามด้วยข้อมูลของคนที่จอง
- **โพลล์:** มีฝ่ายหลัก (master) คับฝ่ายรอง **Select:** ฝ่ายหลักเตือน รอยืนยันพร้อม แล้วส่ง **Poll:** ฝ่ายหลักถามทีละฝ่ายรอง NAK แปลว่าไม่มีอะไรจะส่ง
- **โทเคน:** สถานีเรียงเป็นวงตรรกะ คนถือโทเคนเท่านั้นที่ส่ง แล้วส่งโทเคนต่อ สายจริงไม่จำเป็นต้องเป็นวง

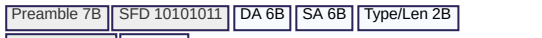
FDMA: แต่ละสถานีได้ย่านความถี่ (FDM ข้างล่าง) เซลลูลาร์ ดาวเทียม **TDMA:** ทั้งย่าน แต่ผลัดกันตามเวลา (TDM ข้างล่าง) **CDMA:** ทุกสถานี ส่งพร้อมกัน ทั้งย่าน แยกกันด้วยรหัส

รหัส Walsh ของ CDMA ตั้งฉากกัน กฎ: ข้อมูล 0 → ส่ง −1 คูณรหัส ข้อมูล 1 → ส่ง +1 คูณรหัส ไม่ว่าง → 0 ผลบวกบนช่องเป็นเวกเตอร์เดียว สถานี i กู้บิตด้วยผลคูณภายในกับรหัสของตน: ผล / N เป็น +1, −1 หรือ 0 ผลคูณภายในของรหัสกับตัวเองคือ N กับรหัสอื่น (หรือส่วนเติมเต็ม) คือ 0

	การเข้าใช้	ใจความ
1G	FDMA แอนะล็อก	เสียงแบบวงจร
2G	TDMA/FDMA, CDMA	เสียงดิจิทัล, SMS
3G	WCDMA / CDMA2000	อินเทอร์เน็ตมือถือ
4G	OFDMA ขาลง, SC-FDMA ขาขึ้น	ทั้งหมดเป็น IP, VoLTE
5G	OFDMA, massive MIMO	eMBB, IoT, หน่วงต่ำ

เฟรม Ethernet

Xerox PARC ปี 1973–74 ได้แรงบันดาลใจจาก ALOHA (Metcalfe) เป็น IEEE 802.3 ปี 1983 แล้วชนะตลาด LAN ไร้การเชื่อมต่อ: ไม่มีการตั้งสาย แต่ละเฟรมอิสระ ไม่เชื่อถือได้: ไม่มี ACK ชั้นลิงก์ ไม่มีโฟลว์คอนโทรลของ LLC คล้าย IP และ UDP ให้ TCP เป็นคนซ่อม



- Preamble (01 สลับ) กับ SFD ถูกเติมโดย PHY เพื่อซิงก์นาฬิกา ไม่นับใน 64 ไบต์ Wireshark ซ่อนสองฟิลด์นี้และ FCS
- เฟรมสั้นสุด **64 B = 512 บิต** เพื่อให้ผู้ส่งยังส่งอยู่ตอนเสียงสะท้อนการชนกลับมา เฮดเดอร์+เทรลเลอร์ = 18 B → เพย์โหลดขั้นต่ำ **46 B** (สั้นกว่านั้นให้แพด)
- เฟรมยาวสุด **1518 B** (ไม่นับ preamble) เพย์โหลดยาวสุด **1500 B** เพราะหน่วยความจำสมัยนั้นแพง และสถานีหนึ่งต้องไม่ยึดสายที่ใช้ร่วม
- Type/length: ค่า ≥ 1536 (0x0600) เป็นชนิด (IPv4 = 0x0800) ค่าน้อยกว่านั้นเป็นความยาว

เพิ่มเติม ติดแท็ก 802.1Q แล้วเฟรมยาวขึ้น 4 ไบต์ (สูงสุด 1522) ความยาวขั้นต่ำ 64 ไบต์เป็นกฎของ CSMA/CD พอร์ตสวิตช์ฟูลดูเพล็กซ์ไม่ต้องใช้มัน เพื่อตรวจการชน แต่ NIC ยังบังคับความยาวนี้

ที่อยู่ MAC

48 บิตบน NIC เขียนเป็นเลขฐานสิบหกคั่นโคลอน (4A:30:10:21:10:1A) 24 บิตแรกคือ OUI (IEEE, ผู้ผลิต) 24 บิตหลังคือซีเรียลของผู้ผลิต ที่อยู่ซ้ำเกิดได้ (โรงงานพลาด หรือ VM) เมื่อนั้นเปลี่ยน NIC หรือแก้ในซอฟต์แวร์

- ส่งทีละไบต์จากซ้ายไปขวา แต่**ในแต่ละไบต์ส่ง LSB ก่อน** บิตที่บอกยูนิคาสต์/มัลติคาสต์จึงไปถึงก่อน
- **ยูนิคาสต์:** I/G = 0 NIC ใบเดียว **มัลติคาสต์:** I/G = 1 (เช่น IGMP) **บรอดคาสต์:** FF:FF:FF:FF:FF:FF DHCP กับ ARP ต้องใช้บรอดคาสต์
- บนสายที่ใช้ร่วม ทุก NIC ได้ยินทุกเฟรม แล้วเก็บเฉพาะของตน บรอดคาสต์หรือมัลติคาสต์ที่ตนเข้ากลุ่ม

ซับเน็ตเดียวกัน: MAC ของเฟรม = โฮสต์ปลายทาง **คนละเครือข่าย:** MAC ของเฟรม = เกตเวย์ (เราเตอร์ปริยาย) IP ในแพ็กเก็ตไม่เปลี่ยน เราเตอร์สร้างเฟรมใหม่

ความเร็วและสวิตช์

- **Ethernet ใช้ร่วมแบบมาตรฐาน:** CSMA/CD แบบ 1-persistent สายยาวได้ประมาณ **2500 ม.**
- **Fast (100 Mbps):** เฟรมสั้นสุดยัง 64 B แต่เร็วขึ้น 10 เท่า สายที่ใช้ร่วมจึงเหลือ **250 ม.** (ในทางปฏิบัติ UTP 100 ม.) Fast Ethernet แบบสวิตช์เป็นฟูลดูเพล็กซ์ ไม่ใช้ CSMA/CD Autonegotiation เลือกความเร็วร่วม มักเป็นฝั่งที่ช้ากว่า
- **Gigabit 802.3z:** รูปแบบเฟรมเดิม มี autonegotiation กรณีฮาล์ฟดูเพล็กซ์เก่า ความยาวขั้นต่ำกลายเป็น 512 B (**carrier extension**) เฟรมสั้นอาจถูกมัดรวม (**frame bursting**) ไม่มีฮับระดับ Gbps สวิตช์ฟูลดูเพล็กซ์ไม่มีการชน จึงไม่ใช้ส่วนขยายนั้น

การส่งต่อ

- ต้นทางที่ไม่รู้จัก → เรียนรู้ **MAC + พอร์ต** ปลายทางที่รู้จัก → ส่งเฉพาะพอร์ตนั้น ปลายทางไม่รู้จัก บรอดคาสต์ หรือมัลติคาสต์ → **flood**
- พอร์ตที่ต่อไปอีกสวิตช์ถือ **MAC ได้หลายตัว**
- **Store-and-forward:** รับทั้งเฟรม ตรวจ CRC แล้วส่งต่อ จำเป็นถ้าจะทำ QoS จากชนิดเฟรม
- **Cut-through:** ส่งต่อเมื่อเห็น DA แล้ว เร็ว ไม่ทำ QoS อาจส่งเฟรมที่ CRC เสีย **Fragment-free** รอ 64 B (หน้าต่างการชนยุคเก่า) แล้วตัดผ่าน
- **Auto-MDIX** รับได้ทั้งสายตรงและสายไขว้ ไม่ใช่ autonegotiation (ความเร็ว/ดูเพล็กซ์)

ARP

บน LAN ท้องถิ่น เฟรมต้องมี MAC ARP แปลง IPv4 → MAC A บรอดคาสต์คำขอ (ช่อง MAC เป้าหมายเป็นศูนย์ทั้งหมด) เจ้าของยูนิคาสต์ตอบกลับ ทั้งสองฝั่งแคชไว้ (`arp -a`) ARP ไม่ข้ามเราเตอร์ เมื่อ IP อยู่ไกล โฮสต์จะ ARP หา **เกตเวย์**

อุปกรณ์และวงวน

อุปกรณ์	ชั้น	พฤติกรรม
รีพีตเตอร์	L1	สร้างบิตใหม่ (ไม่ใช่เครื่องขยาย) ต่อสองเซกเมนต์
ฮับ	L1	ส่งเฟรมออกทุกพอร์ตที่เหลือ collision domain เดียว
บริดจ์	L1+L2	มัก 2 พอร์ต กรองด้วยตาราง ไม่เขียน MAC ใหม่
สวิตช์	L1+L2	บริดจ์หลายพอร์ต เรียนรู้ collision domain ต่อพอร์ต
เราเตอร์	L3	เฟรมใหม่ทุกฮอป แยก broadcast domain

วงวน: สวิตช์สำรองที่ตารางว่างจะ flood เฟรมออกทุกพอร์ต รวมถึงย้อนหากันสำเนาทวีคูณ (พายุบรอดคาสต์) ผู้ดูแลยังอยากได้ลิงก์เกินไว้เป็นทางสำรอง

STP (IEEE 802.1D) เก็บสายจริงไว้ แต่บล็อกบางพอร์ต ให้โทโพโลยีที่ใช้งานเป็นต้นไม้ ใส่ต้นทุนให้แต่ละลิงก์ (ตัวอย่างสไลด์ใช้จำนวนฮอป) เลือก root สวิตช์ที่เหลือเลือก **root port** (ทางที่ดีสุดไป root) แต่ละเซกเมนต์เลือก **designated port** พอร์ตที่เหลือถูก**บล็อก** (ไม่ส่งเฟรมผู้ใช้)

VLAN

ไม่มี VLAN สวิตช์ที่ต่อกันคือ broadcast domain ชั้น 2 ก้อนใหญ่ VLAN คือกลุ่มเชิงตรรกะ แต่ละ VLAN เป็น broadcast domain ของตัวเอง: น้ำท่วมเล็กลง แยกกลุ่มได้ (ความปลอดภัย แผนก) และย้ายพอร์ตได้โดยไม่ย้ายสาย

- **VLAN ปริยาย:** VLAN 1 พอร์ต access ที่ยังไม่กำหนดตกมาที่นี่ อย่าใช้กับผู้ใช้
- **VLAN ข้อมูล** กับ **VLAN เสียง** พอร์ต access มี VLAN ข้อมูลได้หนึ่ง และอาจมี VLAN เสียงได้อีกหนึ่ง โทรศัพท์ VoIP คือสวิตช์ 3 พอร์ตตัวเล็ก CDP บอกโทรศัพท์ว่าต้องแท็ก VLAN เสียงไหน
- **Native VLAN** (เฉพาะทรังก์): เฟรมของ VLAN นี้ไปแบบ**ไม่ติดแท็ก** VLAN อื่นบนทรังก์ 802.1Q ติดแท็กทั้งหมด
- **แท็ก 802.1Q** ยาว 4 ไบต์ แทรกหลัง MAC ต้นทาง: TPID `0x8100` + TCI (priority, DEI, รหัส VLAN 12 บิต) คำนวณ FCS ใหม่
- **พอร์ต access:** VLAN ข้อมูลเดียว ไปหาโฮสต์ปลาย ไม่ติดแท็กบนสาย **ทรังก์:** จุดต่อจุดระหว่างสวิตช์ (หรือสวิตช์กับเราเตอร์) แบกหลาย VLAN
- ช่วงบน Catalyst 2960/3650: ปกติ **1–1005** (เก็บใน `vlan.dat` สร้างในโหมด global) แบบขยาย **1006–4094** มีรหัส VLAN ได้กว่า 4000 แต่สวิตช์ยังมีเพดานจำนวนที่ใช้งานพร้อมกันน้อยกว่านั้น

IOS (ย่อ)

```
vlan 10
name Faculty
interface fa0/18
 switchport mode access
 switchport access vlan 10
 switchport voice vlan 150
interface fa0/1
 switchport mode trunk
 switchport trunk native vlan 99
 switchport trunk allowed vlan 10,20,99
show vlan brief
show interfaces fa0/1 switchport
no vlan 10
```

ก่อน `no vlan` ให้ย้ายพอร์ตสมาชิกไปที่อื่น ไม่งั้นจะตกกลับ VLAN 1 คืนทรังก์ไปทาง access ด้วย `switchport mode access` เราเตอร์บ้าน / NETGEAR: **untagged** = อุปกรณ์ปลาย (access), **tagged** = ทรังก์

เพิ่มเติม Native VLAN ไม่ตรงกัน: ฝั่งหนึ่งแท็กสิ่งที่อีกฝั่งถือว่าเป็น native เฟรมเลยรั่วเข้า VLAN ผิดหรือถูกทิ้ง ให้ native VLAN ตรงกัน และอย่าให้ผู้ใช้ใช้ มัน ARP ไม่ใช่โพรโทคอลหาเส้นทาง สวิตช์ไม่เขียน MAC ในเฟรมใหม่ เราเตอร์เขียน เพราะมันสร้างเฟรมใหม่ VLAN 1 และ VLAN สงวน 1002–1005 ของ token ring/FDDI ลบไม่ได้